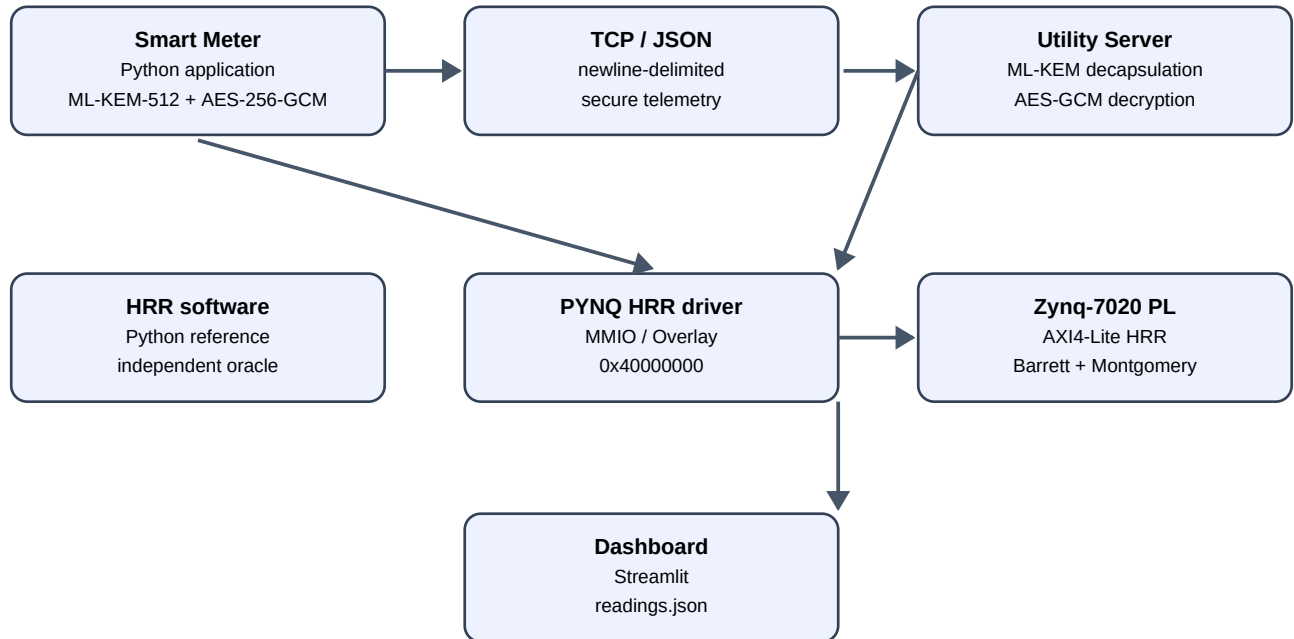


QuantumSecurity

Project Architecture & Implementation Status

Purpose. This document summarizes the supplied QuantumSecurity project as it currently exists, including the software security stack, HRR FPGA accelerator, PYNQ-Z2 architecture, AXI register interface, generated hardware artifacts, and remaining validation work.

SECURE TELEMETRY + FPGA ACCELERATION ARCHITECTURE



Architecture overview

The project combines a smart-meter application, post-quantum key establishment using ML-KEM-512, authenticated telemetry using AES-256-GCM, and a hardware HRR modular-multiplication accelerator implemented in Verilog and exposed to a PYNQ-Z2 through AXI4-Lite.

1. Security and application software

ML-KEM-512: The utility server generates an ML-KEM-512 key pair. The smart meter receives the public key, performs encapsulation, and returns the ciphertext. The server decapsulates the ciphertext to obtain the shared secret. The source implements this flow in the crypto and smart-meter/server modules.

AES-256-GCM: The shared secret is processed with HKDF using an application-specific context string to derive a 32-byte AES key. Telemetry is encrypted with AES-GCM using a 12-byte random nonce; the nonce is carried with the ciphertext for transport.

Transport/application protocol: Smart-meter and server communication uses TCP sockets with newline-delimited JSON messages. Encrypted telemetry is Base64-encoded for JSON transport. The server validates and stores readings in `readings.json`, which is consumed by the dashboard.

2. HRR accelerator

The HRR core computes $r = (a \times b) \bmod 3329$, matching the ML-KEM modulus. The RTL uses Barrett conversion into the Montgomery domain, multiplication, and two Montgomery reductions to return the result to the normal domain. The sequential controller exposes start and done signals and preserves 12-bit operands/results.

Independent reference: `hrr_reference.py` implements the same mathematical operation in Python and is intended as the software test oracle.

Hardware interface: `hrr_axi.v` wraps the accelerator as an AXI4-Lite slave. The wrapper converts the active-low AXI reset to the HRR core's active-high reset and turns a software START write into a one-cycle HRR start pulse.

3. AXI register map

Offset	Register	Bits	Description
0x00	A	[11:0]	Operand A
0x04	B	[11:0]	Operand B
0x08	CONTROL	[0]	START; write 1 to begin; self-clearing pulse
0x0C	STATUS	[0]	DONE; sticky until next START
		[1]	BUSY
0x10	RESULT	[11:0]	Result; valid when DONE=1

The PYNQ driver uses these offsets directly, validates operands against `q=3329`, writes A/B/START, polls STATUS until DONE, and reads the 12-bit result.

4. PYNQ-Z2 hardware architecture

The supplied Vivado build script creates a Zynq-7020/PYNQ-Z2 design with the Processing System 7, enables the PS-to-PL general-purpose AXI master, sets the FPGA peripheral clock to 50 MHz, and connects the master to the HRR AXI4-Lite peripheral using AXI automation.

The current project artifacts include the FPGA bitstream, hardware metadata, and XSA handoff. The AXI peripheral is mapped at the base address established by the generated design; the supplied PYNQ driver can either use the overlay's `hrr_axi_0` metadata or an explicit base address.

The build script also documents why 50 MHz is used: the HRR Montgomery-reduction combinational path was reported as not closing at 100 MHz, while 50 MHz provides timing margin without changing the RTL.

5. PYNQ software path

The supplied `pynq_hrr_driver.py` loads the bitstream with PYNQ Overlay, obtains the HRR MMIO interface, and exposes `fpga_hrr(a, b)`. A self-check covers boundary cases plus randomized cases and compares every FPGA result with the direct modulo-3329 oracle.

Expected hardware test: load the supplied bitstream/metadata on the PYNQ-Z2 and run the driver. A passing self-check demonstrates that the physical FPGA datapath, AXI register interface, PYNQ MMIO path, and result handling are functioning together.

6. Current implementation status

Area	Status	Evidence / current state
ML-KEM-512 key establishment	Complete	Server key generation/encapsulation/decapsulation flow implemented.
AES-256-GCM telemetry	Complete	HKDF derives 32-byte AES key; 12-byte nonce; authenticated encryption.
Smart meter ↔ server	Complete	TCP + newline-delimited JSON protocol implemented.
HRR Python reference	Complete	Barrett conversion + Montgomery reduction reference model.
HRR Verilog core	Complete	Synthesizable modular multiplication for <code>q=3329</code> .
AXI4-Lite wrapper	Complete	Register interface implemented and packaged as IP.
PYNQ-Z2 Vivado design	Complete	PS7 + AXI-Lite HRR block design built.
Bitstream / hardware handoff	Complete	Project package contains .bit, .hwh and .xsa artifacts.
PYNQ MMIO driver	Complete	Driver implements START/DONE/BUSY/result access and self-check.
Physical PYNQ-Z2 test	Pending	Must run the driver against the actual board.
Application HW/SW integration	Pending	Current integration layer defaults to software HRR fallback.

Area	Status	Evidence / current state
End-to-end demonstration	Pending	Final ML-KEM → AES-GCM → application/FPGA path remains to be exercised.

7. Remaining work and acceptance path

Step 1 — Physical FPGA verification. Load the generated overlay on the PYNQ-Z2 and run the supplied HRR self-check. Record successful boundary/random tests and confirm `fpga_hrr(7,9)=63`.

Step 2 — Hardware/software integration. The current application-facing integration module still defaults to the Python HRR reference. The production path should be configured so the application calls the PYNQ accelerator when hardware is available, while retaining the Python reference as an independent test/fallback path.

Step 3 — End-to-end demonstration. Exercise the complete smart-meter workflow: ML-KEM-512 handshake → shared-secret derivation → AES-256-GCM telemetry → server decryption/storage → dashboard, while demonstrating the verified FPGA HRR operation through the hardware interface.

Step 4 — Final documentation. Capture the PYNQ test result, board/bitstream version, register map, software versions, and any measured latency so the implementation is reproducible by a second developer.

8. Project deliverables currently present

Layer	Representative files / artifacts
Cryptography	crypto/mlkem.py; crypto/secure_data.py
Smart meter	smart_meter/smart_meter.py; smart_meter/data_generator.py
Server/dashboard	server/server.py; server/readings.json; dashboard/app.py
HRR reference + RTL	hrr/hrr_reference.py; hrr/hrr.v; hrr/hrr_tb.v; hrr/hrr_test_vectors.txt
AXI wrapper	hrr/hrr_axi.v; hrr/hrr_axi_tb.v
PYNQ hardware	fpga/hrr_bd_wrapper.bit; fpga/hrr_bd_wrapper.hwh; fpga/hrr_pynq_wrapper.xsa
PYNQ software	fpga/pynq_hrr_driver.py
Vivado automation	vivado/hrr/build_hrr_axi_project.tcl
Dependencies	requirements.txt

Source basis: generated from the contents of the supplied `QuantumSecurity.zip`. The summary intentionally reports the project's implemented structure and documented status; physical-board execution has not been inferred from the presence of the bitstream artifacts.